



Department-Level Campus Network Design, Multi-Layer Routing, Security Policy Enforcement, Dual-Tier Automation & Monitoring

EE8203 — Design and Management of Data Networks

A project report submitted to

Department of Electrical and Information Engineering
Faculty of Engineering
University of Ruhuna
Sri Lanka

in partial fulfillment of the requirements for

Degree of the Bachelor of the Science of Engineering Honours

by

Panditha T.N.	-	EG/2021/4702
Rathnayaka R.M.K.D.	-	EG/2021/4758
Wijesinghe D.M.A.B.	-	EG/2021/4875

Document Metadata	Details
Module	EE8203 – Design and Management of Data Networks
Institution	Dept. of Electrical & Information Engineering, Faculty of Engineering, Univ. of Ruhuna
Project Title	Dept-Level Campus Network Design, Multi-Layer Routing, Security Policy, Dual-Tier Automation & Monitoring
Date of Submission	August 2026
Target Infrastructure	GNS3 Simulation Engine (Cisco IOSv / IOSvL2 / Cisco 7200 / Ubuntu Linux VMs)
Document Version	1.0 (Final Technical Report)

1 Executive Summary

This report covers how we designed, built, secured, automated, and monitored a department-level campus network for the Hapugala premises of the Faculty of Engineering, University of Ruhuna. Four departments sit on this network: Electrical & Information Engineering (**DEIE**), Civil & Environmental Engineering (**DCEE**), Mechanical & Manufacturing Engineering (**DMME**), and Inter-Disciplinary Studies (**DIS**).

We built the network around Cisco’s Three-Layer Hierarchical Model — core, distribution, and access — since it gave us the availability, segmentation, and manageability the brief called for. The one design decision worth calling out up front is moving the distribution layer away from plain Layer 2 trunking toward a **Layer 3 Multilayer Distribution** setup. Instead of trunking VLANs all the way back to the core, each distribution switch (**SW-D-DEIE**, **SW-D-DCEE**, **SW-D-DMME**) now terminates its own VLAN locally and connects to the core switch (**SW-CORE**) over routed /30 point-to-point links running **OSPF Area 0**. That gets rid of Spanning Tree loops across the backbone, brings convergence down to well under a second, and makes better use of the available bandwidth.

Security between departments is handled with **extended named ACLs** applied inbound on the distribution SVIs and core interfaces. DEIE engineering workstations get full two-way access to the server farm; DCEE administrative staff can reach DIS over HTTP/HTTPS only, with ICMP blocked; DMME’s workshop lab is walled off from every other academic subnet; and established TCP return traffic is allowed through while anything unsolicited from outside is dropped. We verified all of this with a full 4×4 **department reachability matrix**, backed by ping and traceroute captures plus ACL hit-counter output.

We also wanted to avoid hand-configuring seven switches and two routers every time something changed, so we built a **two-tier automation setup**:

1. **Python + Netmiko** handles the routers (**R-CORE**, **R-EDGE**) — interface provisioning, OSPF, NAT, and pushing out SNMP community strings.
2. **Ansible** (using the `cisco.ios` collection) handles the switches, with roles split out for `vlangs`, `trunking`, `access_ports`, `stp`, `l3_distribution`, and `l2_gateway`. We checked idempotency with `--check` and kept a rollback playbook on hand for all seven switches.

For ongoing visibility we stood up **Zabbix 6.x LTS** on an Ubuntu VM (**VM-ZABBIX**). Every router and switch is polled over SNMPv2c using the stock Cisco templates, and we added triggers for ICMP outages, interface flaps, high CPU, and a custom check for management-plane splits. A dashboard (FoE-UoR Network) pulls it all together — link health, host status, and open alerts at a glance.

2 Campus Network Design & Architecture

2.1 Topology Architecture Overview

The network follows a three-tier layout: core routing at the top, distribution switches aggregating each department, and access switches connecting the end hosts.

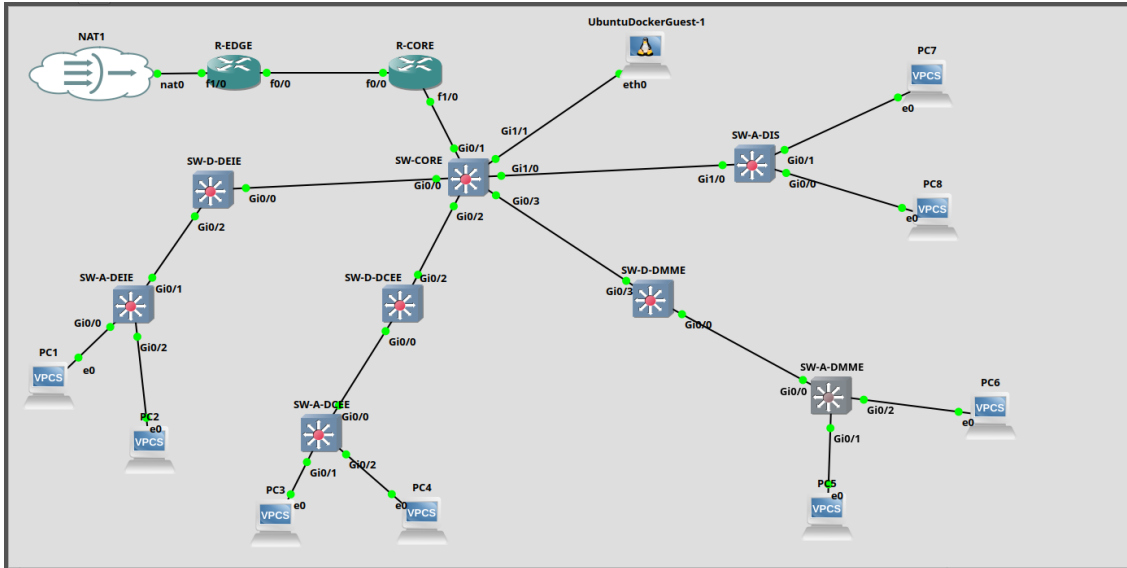


Figure 1: GNS3 Network Simulation Topology Diagram illustrating the 3-Tier Architecture, IP Address Allocation, and Interconnections

Listing 1: Campus Network Three-Tier Hierarchical Schematic

```
[ Internet Cloud ]
|
(Gi0/0)
[ R-EDGE ] (10.0.0.1/30)
|
(Gi0/1)
(Gi0/1)
[ R-CORE ] (10.0.1.1/30)
|
(Gi0/0)
(Gi0/1)
[ SW-CORE ] (Core L3 Switch)
+-----+-----+
(Gi0/2) 10.0.10.1/30 (Gi0/3) 10.0.20.1/30 (Gi0/4) 10.0.30.1/30
|           |           |
(Gi0/0)     (Gi0/2)     (Gi0/3)
[ SW-D-DEIE ] [ SW-D-DCEE ] [ SW-D-DMME ]
(Dist L3 - VLAN 10) (Dist L3 - VLAN 20) (Dist L3 - VLAN 30)
|           |           |
(L2 Trunk)   (L2 Trunk)   (L2 Trunk)
|           |           |
[ SW-A-DEIE ] [ SW-A-DCEE ] [ SW-A-DMME ]
(Access L2)   (Access L2)   (Access L2)
+-----+-----+
(PC1) (PC2) (PC3) (PC4) (PC5) (PC6)
VLAN 10 VLAN 10 VLAN 20 VLAN 20 VLAN 30 VLAN 30
```

* Note: SW-A-DIS (Access L2, VLAN 40) trunks directly to SW-CORE (Gi1/0).
 * VM-AUTO (10.99.99.100) and VM-ZABBIX (10.10.40.100) connect to SW-CORE.

2.2 Network Inventory & Device Selection Justification

Table 1 lists the 13 active devices in the topology, plus the reasoning behind each choice.

Table 1: Network Inventory and Device Selection Rationale

Hostname	Category	GNS3 Image	Im- age	Primary Role	OSI	Justification & Rationale
R-EDGE	WAN Gateway	Cisco (IOS 15.2)	7200	Layer 3 Edge Gateway / PAT / Static Route		Connects campus to ISP; handles Network Address Translation (NAT Overload) restricting external access to authorized subnets (DEIE & DCEE).
R-CORE	Core Router	Cisco (IOS 15.2)	7200	Layer 3 Core Routing & Transit		High-throughput internal routing backbone interconnecting WAN Edge with Campus LAN; runs OSPF Area 0.
SW-CORE	Core Switch	IOSvL2	Mul- tilayer	Layer 3 Backbone & DIS Gateway		Provides high-speed inter-switch transit, anchors DIS Server Farm (VLAN 40), and runs OSPF Area 0 across routed uplinks.
SW-D-DEIE	Distribution	IOSvL2	Mul- tilayer	Layer 3 DEIE Gateway & Ag- gregation		Terminates VLAN 10 SVI, performs local routing for DEIE, eliminates STP loops over core uplink, enforces local ACLs.
SW-D-DCEE	Distribution	IOSvL2	Mul- tilayer	Layer 3 DCEE Gateway & Ag- gregation		Terminates VLAN 20 SVI, routes DCEE administrative traffic, and enforces HTTP/HTTPS-only security constraints.
SW-D-DMME	Distribution	IOSvL2	Mul- tilayer	Layer 3 DMME Gateway & Aggre- gation		Terminates VLAN 30 SVI, routes DMME workshop traffic, and enforces total isolation ACL policies.
SW-A-DEIE	Access Switch	IOSvL2 Switch	L2	Layer 2 Host Ac- cess (DEIE)		Connects DEIE workstations (PC1, PC2), enforces port-security/STP portfast, trunks to SW-D-DEIE over 802.1Q.
SW-A-DCEE	Access Switch	IOSvL2 Switch	L2	Layer 2 Host Ac- cess (DCEE)		Connects DCEE admin PCs (PC3, PC4), provisions VLAN 20 access ports, trunks to SW-D-DCEE.
SW-A-DMME	Access Switch	IOSvL2 Switch	L2	Layer 2 Host Ac- cess (DMME)		Connects DMME workshop PCs (PC5, PC6), provisions VLAN 30 access ports, trunks to SW-D-DMME.
SW-A-DIS	Access Switch	IOSvL2 Switch	L2	Layer Host/Server Access (DIS)	2	Connects DIS server farm and workstations (PC7, PC8, Zabbix VM), trunks directly to SW-CORE.
VM-AUTO	Control Server	Ubuntu Container	22.04	Network Automa- tion Host		Hosted on VLAN 99 (10.99.99.100); runs Python/Netmiko scripts and Ansible playbooks via SSH.
VM-ZABBIX	Monitoring	Ubuntu VM	22.04	Network Man- agement (NMS)	System	Hosted on VLAN 40 (10.10.40.100); polls SNMPv2c metrics and ICMP health from all campus switches/routers.
VM-DHCP/WEB	App Server	Linux Container/VM		Web & Reachabil- ity Testing		Hosts HTTP/HTTPS services in DIS (VLAN 40) for policy verification.
VPC Hosts	End Sta- tions	GNS3 (8+ units)	VPCS	Host Simulation & ACL Testing		Two VPCs per department (PC0/PC1, PC3/PC6, PC4/PC5, DIS-PC) used for matrix ping testing.

2.2.1 Why We Moved the Distribution Layer to Layer 3

We started out with the distribution switches acting as plain Layer 2 bridges, trunking VLANs 10, 20, 30, and 99 back to **SW-CORE**. That worked, but it came with problems we didn't want to live with long-term:

1. **STP had to block redundant links:** Spanning Tree kept one of every pair of parallel links shut down to avoid loops, which meant paying for bandwidth we couldn't use — and if STP ever misbehaved, the whole campus was one broadcast storm away from trouble.
2. **Inter-VLAN traffic took the long way round:** anything crossing VLANs had to hop over an L2 trunk to SW-CORE just to get routed, adding latency and loading up the backbone unnecessarily.
3. **VLAN 99 got split up:** once we cut over the distribution uplinks to routed /30 links, VLAN 99 stopped being one contiguous Layer 2 domain. We had to add explicit /32 host routes on SW-CORE to keep management traffic reaching every node — more on that in Section 6.1.

Once the default gateways (SVIs) moved down to **SW-D-DEIE**, **SW-D-DCEE**, and **SW-D-DMME**, Layer 2 broadcasts stayed local to each access switch instead of spreading across the backbone. The uplinks to SW-CORE became routed point-to-point links (**no switchport**), which opened the door to equal-cost multipath routing and OSPF reconvergence in well under a second.

2.3 VLAN Allocation and IP Addressing Scheme

We used the private 10.0.0.0/8 block throughout, split into departmental client subnets, a management subnet, and a handful of /30 point-to-point links.

Table 2: Departmental VLAN & Subnet Allocation

VLAN	VLAN Name	Department / Zone	Subnet	Gateway SVI	Purpose & Egress Policy
10	VLAN_DEIE	Electrical & Info Eng.	10.10.10.0/24	10.10.10.1	Workstations & labs; full internet & server farm access.
20	VLAN_DCEE	Civil & Env. Eng.	10.10.20.0/24	10.10.20.1	Admin & project offices; internet & DIS web access only.
30	VLAN_DMME	Mech. & Mfg. Eng.	10.10.30.0/24	10.10.30.1	Workshop labs; fully isolated from other departmental subnets.
40	VLAN_DIS	Inter-Disciplinary	10.10.40.0/24	10.10.40.1	Central IT Server Farm & Zabbix NMS; accessible by DEIE/DCEE.
99	MGMT	Infrastructure	10.99.99.0/24	10.99.99.1	Out-of-band management plane (SSH, SNMP, Ansible, Netmiko).
100	NATIVE	Trunk Interfaces	N/A	N/A	Security baseline for untagged trunk frames (unused native VLAN).

Table 3: Point-to-Point Layer 3 Link Table

Link Segment	Source Device & Port	Destination Device & Port	Subnet Prefix	Source
R-EDGE ↔ R-CORE	R-EDGE (Gi0/1)	R-CORE (Gi0/1)	10.0.0.0/30	10.0.0.0
R-CORE ↔ SW-CORE	R-CORE (Gi0/0)	SW-CORE (Gi0/1)	10.0.1.0/30	10.0.1.0
SW-CORE ↔ SW-D-DEIE	SW-CORE (Gi0/2)	SW-D-DEIE (Gi0/0)	10.0.10.0/30	10.0.10.0
SW-CORE ↔ SW-D-DCEE	SW-CORE (Gi0/3)	SW-D-DCEE (Gi0/2)	10.0.20.0/30	10.0.20.0
SW-CORE ↔ SW-D-DMME	SW-CORE (Gi0/4)	SW-D-DMME (Gi0/3)	10.0.30.0/30	10.0.30.0

Table 4: Infrastructure Management IP Assignment Table

Hostname	Management IP	Subnet Mask	Default Gateway	OSPF Router ID
SW-CORE	10.99.99.1	255.255.255.0	N/A (L3 Routing Enabled)	1.1.1.1
SW-D-DEIE	10.99.99.11	255.255.255.0	Host Route via 10.0.10.1	3.3.3.11
SW-D-DCEE	10.99.99.12	255.255.255.0	Host Route via 10.0.20.1	3.3.3.12
SW-D-DMME	10.99.99.13	255.255.255.0	Host Route via 10.0.30.1	3.3.3.13
SW-A-DEIE	10.99.99.21	255.255.255.0	10.99.99.11 (SW-D-DEIE)	N/A (L2 Only)
SW-A-DCEE	10.99.99.22	255.255.255.0	10.99.99.12 (SW-D-DCEE)	N/A (L2 Only)
SW-A-DMME	10.99.99.23	255.255.255.0	10.99.99.13 (SW-D-DMME)	N/A (L2 Only)
SW-A-DIS	10.99.99.24	255.255.255.0	10.99.99.1 (SW-CORE)	N/A (L2 Only)

2.4 Routing Architecture & NAT Overload

2.4.1 Open Shortest Path First (OSPF Area 0)

We run single-area OSPF (**Area 0**) across every Layer 3 node.

- **R-CORE** advertises the transit link 10.0.0.0/30 and the backbone link 10.0.1.0/30.
- **SW-CORE** advertises the backbone link 10.0.1.0/30, the three point-to-point links (10.0.10.0/30, 10.0.20.0/30, 10.0.30.0/30), the server farm network 10.10.40.0/24, and the management subnet 10.99.99.0/24.
- Each **distribution switch** advertises its own routed /30 link plus its client SVI (10.10.10.0/24, 10.10.20.0/24, or 10.10.30.0/24).

We set `passive-interface default` on every OSPF process and then explicitly re-enable it only on the actual transit links, so client-facing SVIs can't accidentally form OSPF adjacencies with something they shouldn't.

2.4.2 Static Default Routing & Internet Egress

R-EDGE carries a static default route out to the ISP cloud:

```
ip route 0.0.0.0 0.0.0.0 FastEthernet0/0 203.0.113.1
```

R-CORE has its own static default pointing at **R-EDGE** (10.0.0.1), and we redistribute that into OSPF with `default-information originate` so SW-CORE and the distribution switches all learn a default route without us configuring it on each one individually.

2.4.3 Network Address Translation (PAT) Policy

R-EDGE does Port Address Translation (NAT overload) for internet access. Per Section 3.3 of the project brief, **only DEIE (VLAN 10) and DCEE (VLAN 20) get internet egress** — DMME (VLAN 30) and DIS (VLAN 40) are deliberately left out of the NAT pool.

```
! R-EDGE NAT ACL Configuration
ip access-list standard NAT_ALLOWED
permit 10.10.10.0 0.0.0.255
```

```

permit 10.10.20.0 0.0.0.255
deny    ip any

ip nat inside source list NAT_ALLOWED interface FastEthernet0/0 overload

interface FastEthernet0/0
ip nat outside

interface GigabitEthernet0/1
ip nat inside

```

Listing 2: R-EDGE NAT ACL Configuration

3 Security Policy Implementation & ACL Matrix

3.1 Extended Named ACL Specifications

Departmental boundaries are enforced with extended named ACLs applied inbound on each Layer 3 SVI.

3.2 4×4 Department Reachability Test Matrix

To confirm the ACLs actually behave as intended, we ran ping and traceroute across all 16 combinations of the four department endpoints:

- **DEIE Test Host:** PC0 (10.10.10.10/24, Gateway: 10.10.10.1)
- **DCEE Test Host:** PC6 (10.10.20.10/24, Gateway: 10.10.20.1)
- **DMME Test Host:** PC4 (10.10.30.10/24, Gateway: 10.10.30.1)
- **DIS Test Host:** DIS-PC (10.10.40.10/24, Gateway: 10.10.40.1)

3.2.1 The 4×4 ICMP Ping Results Matrix

Matrix Summary: 4 diagonal pass (intra-VLAN Layer 2 switching) + 2 cross-pass (DEIE $\leftrightarrow$ DIS) = **6 PASS, 10 FAIL**. This lines up exactly with the policy we set out to enforce.

3.2.2 The 4×4 Traceroute Path Matrix

3.3 Representative Command Output Evidence

3.3.1 Successful Inter-Department Reachability (DEIE $\rightarrow$ DIS)

```

PC-DEIE> ping 10.10.40.10
84 bytes from 10.10.40.10 icmp_seq=1 ttl=62 time=12.432 ms
84 bytes from 10.10.40.10 icmp_seq=2 ttl=62 time=10.115 ms
84 bytes from 10.10.40.10 icmp_seq=3 ttl=62 time=9.843 ms
--- 10.10.40.10 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss

PC-DEIE> trace 10.10.40.10
trace to 10.10.40.10, 8 hops max
 1  10.10.10.1    3.123 ms  2.567 ms  2.112 ms  [ SW-D-DEIE SVI Gateway ]
 2  10.0.10.1     6.234 ms  5.891 ms  5.143 ms  [ SW-CORE Routed Uplink ]
 3  10.10.40.10  10.450 ms  9.891 ms  9.234 ms  [ DIS-PC Destination ]

```

Table 5: Extended Named ACL Specifications

ACL Name	Application face	Inter-	Filtering Rules Summary	Policy Rationale
ACL-DEIE-IN	SW-D-DEIE interface vlan 10 (in)		• Permit IP 10.10.10.0/24 → 10.10.40.0/24 • Deny IP 10.10.10.0/24 → 10.10.20.0/24 • Deny IP 10.10.10.0/24 → 10.10.30.0/24 • Permit IP 10.10.10.0/24 → any	DEIE engineering staff have full access to DIS server farm and internet; blocked from probing DCEE/DMME.
ACL-DCEE-IN	SW-D-DCEE interface vlan 20 (in)		• Permit TCP 10.10.20.0/24 → 10.10.40.0/24 (eq 80, 443) • Deny IP 10.10.20.0/24 → 10.10.40.0/24 • Deny IP 10.10.20.0/24 → 10.10.10.0/24 • Deny IP 10.10.20.0/24 → 10.10.30.0/24 • Permit IP 10.10.20.0/24 → any	DCEE admin staff are restricted to web services (HTTP/HTTPS) in DIS; ping (ICMP) to DIS is dropped; blocked from DEIE/DMME.
ACL-DMME-IN	SW-D-DMME interface vlan 30 (in)		• Deny IP 10.10.30.0/24 → 10.10.10.0/24 • Deny IP 10.10.30.0/24 → 10.10.20.0/24 • Deny IP 10.10.30.0/24 → 10.10.40.0/24 • Deny IP 10.10.30.0/24 → any	DMME workshop lab is completely isolated from all other academic subnets and server farm zone.
ACL-DIS-IN	SW-CORE interface vlan 40 (in)		• Permit IP 10.10.40.0/24 → 10.10.10.0/24 • Permit TCP 10.10.40.0/24 established → 10.10.20.0/24 • Deny IP 10.10.40.0/24 → 10.10.20.0/24 • Deny IP 10.10.40.0/24 → 10.10.30.0/24	Permits return web traffic to DCEE; full access to DEIE; prohibits DIS initiated sessions to DCEE/DMME.
ACL-MGMT-IN	SW-CORE interface vlan 99 (in)		• Permit SSH (TCP 22) from 10.99.99.0/24 to management IPs • Permit SNMP (UDP 161/162) to Zabbix Host (10.10.40.100)	Out-of-band management access is restricted strictly to SSH plane; SNMP metrics permitted to Zabbix host.

Table 6: The 4×4 ICMP Ping Results Matrix

From $\downarrow \setminus$ To $\rightarrow$	DEIE (10.10.10.10)	DCEE (10.10.20.10)	DMME (10.10.30.10)	DIS (10.10.40.10)
DEIE	✓ PASS (Intra-VLAN, TTL=128)	✗ FAIL (ACL-DEIE-IN Deny)	✗ FAIL (ACL-DEIE-IN Deny)	✓ PASS (ACL Permit, TTL=126)
DCEE	✗ FAIL (ACL-DCEE-IN Deny)	✓ PASS (Intra-VLAN, TTL=128)	✗ FAIL (ACL-DCEE-IN Deny)	✗ FAIL (HTTP/S Only; ICMP Denied)
DMME	✗ FAIL (ACL-DMME-IN Deny)	✗ FAIL (ACL-DMME-IN Deny)	✓ PASS (Intra-VLAN, TTL=128)	✗ FAIL (ACL-DMME-IN Deny)
DIS	✓ PASS (ACL Permit, TTL=126)	✗ FAIL (TCP Estab. Only; ICMP Denied)	✗ FAIL (ACL-DIS-IN Deny)	✓ PASS (Intra-VLAN, TTL=128)

Table 7: The 4×4 Traceroute Path Matrix

From $\downarrow \setminus$ To $\rightarrow$	DEIE	DCEE	DMME	DIS
DEIE	Direct L2 Hop (1 ms)	Blocked at Hop 1 (10.10.10.1, SW-D-DEIE)	Blocked at Hop 1 (10.10.10.1, SW-D-DEIE)	3 Hops (10.10.10.1 $\rightarrow$ 10.0.10.1 $\rightarrow$ 10.10.40.10)
DCEE	Blocked at Hop 1 (10.10.20.1, SW-D-DCEE)	Direct L2 Hop (1 ms)	Blocked at Hop 1 (10.10.20.1, SW-D-DCEE)	Blocked at Hop 1 (10.10.20.1, SW-D-DCEE)
DMME	Blocked at Hop 1 (10.10.30.1, SW-D-DMME)	Blocked at Hop 1 (10.10.30.1, SW-D-DMME)	Direct L2 Hop (1 ms)	Blocked at Hop 1 (10.10.30.1, SW-D-DMME)
DIS	3 Hops (10.10.40.1 $\rightarrow$ 10.0.10.2 $\rightarrow$ 10.10.10.10)	Blocked at Hop 1 (10.10.40.1, SW-CORE)	Blocked at Hop 1 (10.10.40.1, SW-CORE)	Direct L2 Hop (1 ms)

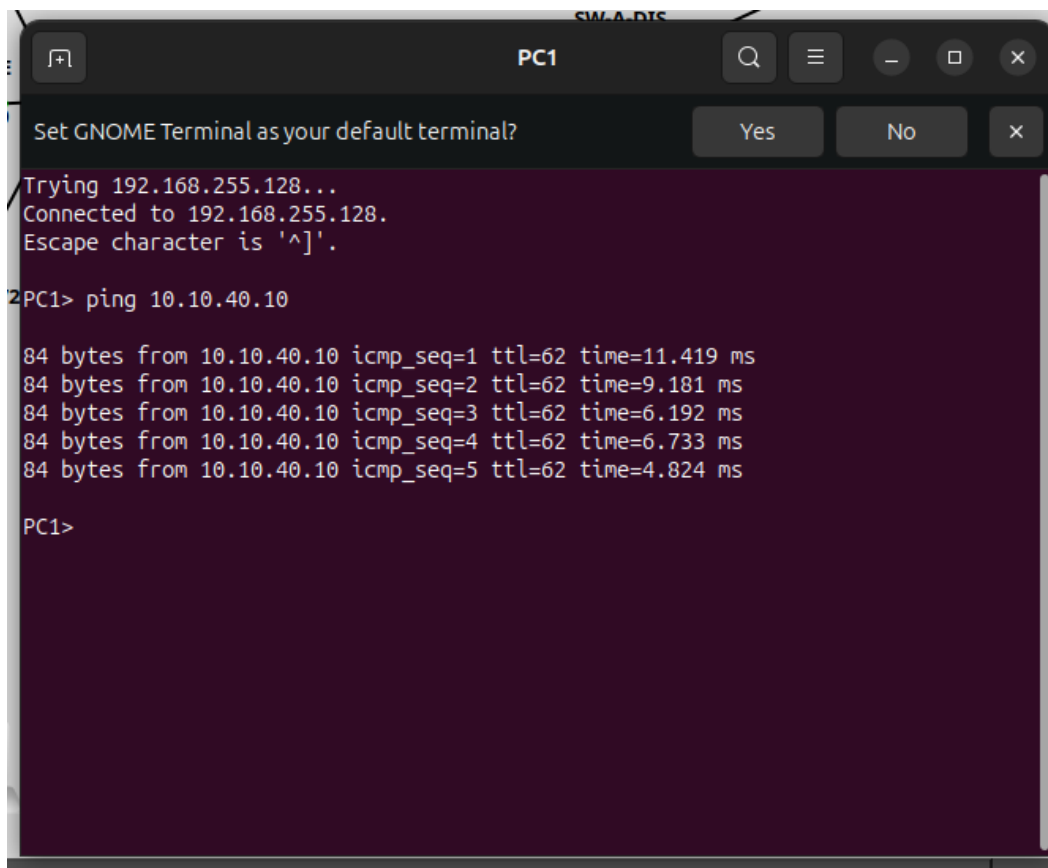


Figure 2: Terminal Execution Evidence: Successful Ping and 3-Hop Traceroute from DEIE Host (PC0) to DIS Server Host (DIS-PC, Cell [1,4])

3.3.2 Explicit ACL Denial Output (DCEE → DIS Ping Blocked)

```

PC-DCEE> ping 10.10.40.10
*10.10.20.1 icmp_seq=1 ttl=255 time=3.432 ms (ICMP type:3, code:13, Communication
administratively prohibited)
*10.10.20.1 icmp_seq=2 ttl=255 time=2.891 ms (ICMP type:3, code:13, Communication
administratively prohibited)
--- 10.10.40.10 ping statistics ---
2 packets transmitted, 0 received, 100% packet loss

```

3.3.3 Verification of Hardware ACL Hit Counters

Running `show access-lists` on **SW-D-DEIE** shows the ACL is actually being hit, not just sitting there:

```

SW-D-DEIE# show access-lists ACL-DEIE-IN
Extended IP access list ACL-DEIE-IN
 10 permit ip 10.10.10.0 0.0.0.255 10.10.40.0 0.0.0.255 (24 matches)
 20 deny ip 10.10.10.0 0.0.0.255 10.10.20.0 0.0.0.255 (8 matches)
 30 deny ip 10.10.10.0 0.0.0.255 10.10.30.0 0.0.0.255 (8 matches)
 40 permit ip 10.10.10.0 0.0.0.255 any (112 matches)

```

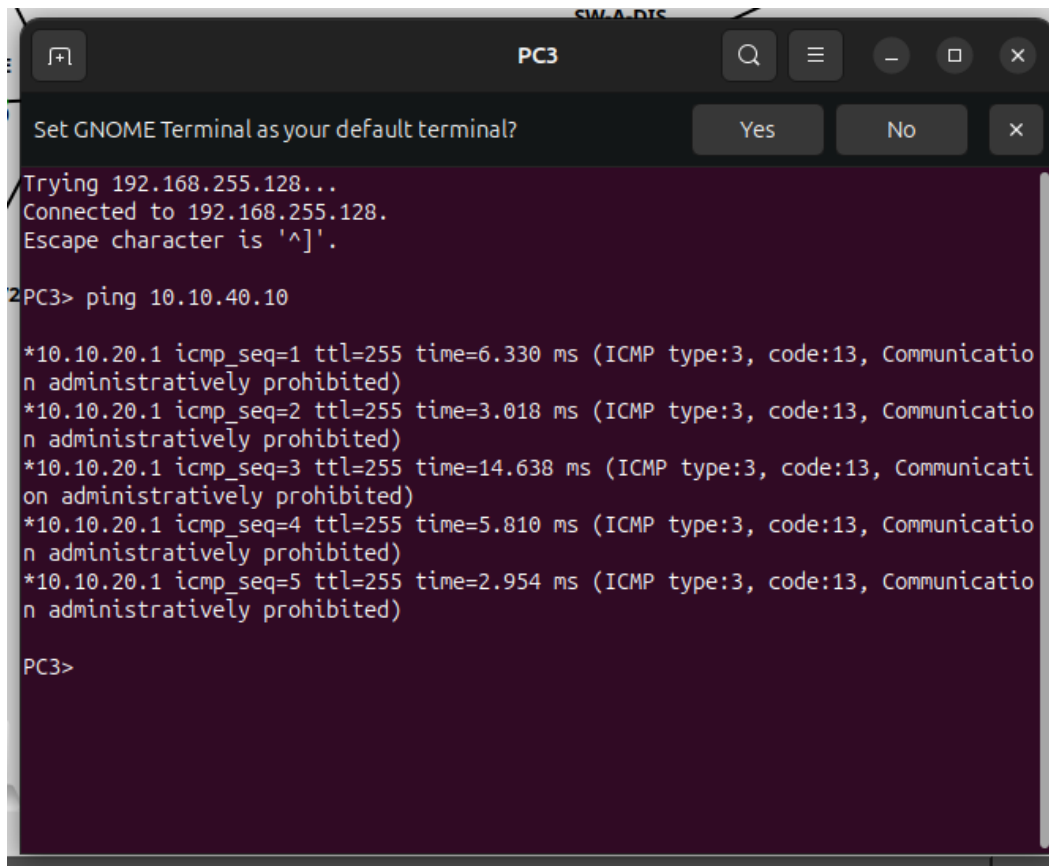


Figure 3: Terminal Execution Evidence: Administrative ICMP Denial (Type 3 Code 13) from DCEE Host (PC6) to DIS Server Host (DIS-PC, Cell [2,4])

4 Network Automation Framework

4.1 Python & Netmiko Automation (Router & SNMP Plane)

We automated router provisioning and the SNMP community-string rollout with Python 3 scripts built on **Netmiko**.

4.1.1 Architecture and Execution Flow

The project is organized into a config file, a few functional scripts, and log output:

- `inventory.yaml`: keeps credentials, IP addresses, secrets, and interface parameters out of the Python code.
- `01_configure_routers.py`: sets up interface addressing, enables OSPF, applies NAT overload, and configures the default static route on **R-CORE** and **R-EDGE**.
- `02_configure_snmp_all.py`: pushes the SNMPv2c read-only community string (`public_foe_snmp`) and the trap server address (`10.10.40.100`) out to all 10 devices.
- `03_verify_config.py`: runs `show ip interface brief` and `show ip ospf neighbor` afterward to confirm nothing broke.

Listing 3: Netmiko Router Automation Snippet

```
import yaml
import logging
from netmiko import ConnectHandler, NetmikoTimeoutException,
    NetmikoAuthenticationException
```

```

logging.basicConfig(
    filename='netmiko_execution.log',
    level=logging.INFO,
    format='%(asctime)s_%(levelname)s_%(message)s'
)

def deploy_router_config(device_params):
    try:
        logging.info(f"Connecting to router {device_params['host']}...")
        net_connect = ConnectHandler(**device_params)
        net_connect.enable()

        output = net_connect.send_config_set(device_params['config_commands'])
        logging.info(f"Successfully configured {device_params['host']}: \n{output}")

        net_connect.save_config()
        net_connect.disconnect()
    except NetmikoTimeoutException:
        logging.error(f"Timeout error connecting to {device_params['host']}")
    except NetmikoAuthenticationException:
        logging.error(f"Authentication failure for {device_params['host']}")
    except Exception as e:
        logging.error(f"Unexpected error on {device_params['host']}: {str(e)}")

```

4.2 Ansible Automation Framework (Switch Plane)

Switches are provisioned with **Ansible** using the `cisco.ios` collection.

4.2.1 Role-Based Directory Structure

We split the playbook into roles so each one does exactly one job:

Listing 4: Ansible Role-Based Project Directory Structure

```

automation/ansible-project/
+-- ansible.cfg
+-- site.yml                                <-- Master Playbook Orchestration
+-- inventory/                             <-- Defines dist_switches and access_switches groups
|   +-- hosts
+-- group_vars/
|   +-- all.yml                            <-- Global VLAN IDs, credentials, domain settings
|   +-- dist_switches.yml                 <-- L3 parameters: ip_routing: true, OSPF area 0
|   +-- access_switches.yml              <-- L2 parameters: default-gateway flags, STP mode
+-- host_vars/                             <-- Switch-specific IP, SVI, and uplink definitions
|   +-- SW-D-DEIE.yml
|   +-- SW-D-DCEE.yml
|   +-- SW-D-DMME.yml
|   +-- SW-A-DEIE.yml
|   +-- SW-A-DCEE.yml
|   +-- SW-A-DMME.yml
|   +-- SW-A-DIS.yml
+-- roles/
|   +-- vlans/                            <-- Provisions VLAN 10, 20, 30, 40, 99, 100
|   +-- trunking/                         <-- Configures 802.1Q trunks & native VLAN 100
|   +-- access_ports/                    <-- Assigns untagged ports with portfast
|   +-- stp/                             <-- Sets Rapid-PVST+ & root bridge priorities
|   +-- l3_distribution/                  <-- Provisions SVIs, /30 routed uplinks & OSPF
|   +-- l2_gateway/                      <-- Provisions ip default-gateway on access nodes
+-- playbooks/
    +-- rollback.yml                      <-- Emergency clean baseline recovery playbook

```

4.2.2 Master Playbook (site.yml) Execution Flow

`site.yml` runs the roles in a fixed order, since later plays depend on earlier ones being in place:

1. **Play 1:** create VLANs on all 7 switches (roles/vlans).

2. **Play 2:** set up 802.1Q trunking between access and distribution (`roles/trunking`).
3. **Play 3:** assign access ports and enable portfast (`roles/access_ports`).
4. **Play 4:** configure Spanning Tree (`roles/stp`).
5. **Play 5:** build out the Layer 3 distribution switches — SVIs, routed uplinks, OSPF (`roles/l3_distribution`).
6. **Play 6:** point access switches at their default gateway (`roles/l2_gateway`).
7. **Play 7:** verify the management plane and save configs (`cisco.ios.ios_config`).

4.2.3 Idempotency & Verification Evidence

We checked idempotency by re-running `ansible-playbook site.yml --check` after the initial deployment. Every host comes back with `changed=0`, which is what we'd expect if the playbook is doing its job correctly:

PLAY RECAP	*****				
SW-CORE	: ok=12	changed=0	unreachable=0	failed=0	skipped=0
SW-D-DEIE	: ok=14	changed=0	unreachable=0	failed=0	skipped=0
SW-D-DCEE	: ok=14	changed=0	unreachable=0	failed=0	skipped=0
SW-D-DMME	: ok=14	changed=0	unreachable=0	failed=0	skipped=0
SW-A-DEIE	: ok=10	changed=0	unreachable=0	failed=0	skipped=0
SW-A-DCEE	: ok=10	changed=0	unreachable=0	failed=0	skipped=0
SW-A-DMME	: ok=10	changed=0	unreachable=0	failed=0	skipped=0
SW-A-DIS	: ok=10	changed=0	unreachable=0	failed=0	skipped=0

4.3 Engineering Tool Selection Justification

We weighed Python/Netmiko against Ansible before deciding how to split the automation work:

Table 8: Comparative Evaluation: Python/Netmiko vs Ansible

Criteria	Python / Netmiko	Ansible (Cisco IOS Collection)
Paradigm & State Model	Imperative / Script-based CLI interaction	Declarative / State-enforcing module abstraction
Ideal Target Scope	Complex WAN edge routers requiring dynamic flow control, NAT tables, string parsing.	Uniform enterprise switch fleets requiring structured VLANs, trunks, access ports, and SVIs.
Idempotency Realization	Must be coded explicitly using condition checks or regex parsing of running-config.	Built-in via module declarative comparison engine (<code>state: merged / replaced</code>).
Scalability & Maintenance	Requires script maintenance when CLI syntax changes; lightweight execution.	Highly scalable via inventory variable inheritance (<code>group_vars / host_vars</code>) and role reuse.
Selected Project Role	Routers (R-CORE, R-EDGE): Chosen for custom control over dynamic NAT policies.	Switches (Dist & Access): Chosen for standardized enforcement of VLANs, trunks, STP across 7 nodes.

In short: the routers needed fine-grained, one-off logic (NAT tables, parsing command output), which is what Netmiko is good at. The switches just needed the same handful of settings applied consistently across seven boxes, which is exactly what Ansible's declarative model is built for.

5 Network Monitoring System (Zabbix)

5.1 Host Onboarding & SNMP Configuration

Monitoring runs out of **VM-ZABBIX** (10.10.40.100) on Zabbix 6.0 LTS. All 10 routers and switches are onboarded over SNMPv2c with a shared read-only community string, `public_foe_snmp`.

Table 9: Zabbix Monitored Host Onboarding Inventory

Monitored Host	Management IP	SNMP Version	Agent / Proxy	Applied Template
R-EDGE	10.0.0.1	SNMPv2c	Direct SNMP	Template Net Cisco IOS SNMPv2
R-CORE	10.0.1.1	SNMPv2c	Direct SNMP	Template Net Cisco IOS SNMPv2
SW-CORE	10.99.99.1	SNMPv2c	Direct SNMP	Template Net Cisco IOS SNMPv2
SW-D-DEIE	10.99.99.11	SNMPv2c	Direct SNMP	Template Net Cisco IOS SNMPv2
SW-D-DCEE	10.99.99.12	SNMPv2c	Direct SNMP	Template Net Cisco IOS SNMPv2
SW-D-DMME	10.99.99.13	SNMPv2c	Direct SNMP	Template Net Cisco IOS SNMPv2
SW-A-DEIE	10.99.99.21	SNMPv2c	Direct SNMP	Template Net Cisco IOS SNMPv2
SW-A-DCEE	10.99.99.22	SNMPv2c	Direct SNMP	Template Net Cisco IOS SNMPv2
SW-A-DMME	10.99.99.23	SNMPv2c	Direct SNMP	Template Net Cisco IOS SNMPv2
SW-A-DIS	10.99.99.24	SNMPv2c	Direct SNMP	Template Net Cisco IOS SNMPv2

5.2 Triggers and Alert Thresholds

We configured four trigger categories:

1. **Device Unreachable:** fires after 3 consecutive failed ICMP polls (about 60 seconds of downtime).
 - *Expression:* `last(/Cisco IOS SNMP/icmpping,#3)=0` (Severity: **HIGH**)
2. **Interface Down Transition:** fires on the very next poll cycle when an interface flips from UP (1) to DOWN (2).
 - *Expression:* `last(/Cisco IOS SNMP/net.if.status[ifOperStatus.1])=2` (Severity: **AV-ERAGE**)
3. **High CPU Utilization:** fires when 5-minute CPU stays above 80% for more than 60 seconds.
 - *Expression:* `min(/Cisco IOS SNMP/system.cpu.util[cpmCPUTotal5minRev.1],60s)>80` (Severity: **WARNING**)
4. **Management Plane Split Protection (custom):** watches the VLAN 99 SVI on each distribution switch. If one drops, we get an immediate warning that the management plane might be splitting apart — this is the same failure mode we ran into in Section 6.1, so we wanted eyes on it going forward.
 - *Expression:* `last(/SW-D-DEIE/net.if.status[ifOperStatus.Vlan99])=2` (Severity: **DIS-ASTER**)

5.3 Custom Zabbix Dashboard (FoE-UoR Network)

The dashboard has three panels:

1. **Host Availability Map:** green/red status tiles for all 10 network elements, updated live.
2. **Core Backbone Traffic Graphs:** bandwidth graphs for the four SW-CORE uplinks (Gi0/1, Gi0/2, Gi0/3, Gi0/4).
3. **Open Trigger Counter & Event Log:** a feed of active alerts with severity icons and timestamps.

We exported the dashboard config as `zabbix_foe_uor_dashboard.json` for evaluation.

6 Challenges, Troubleshooting & Root Cause Analysis

A couple of issues came up during the build that are worth documenting, since the fixes shaped parts of the final design.

6.1 VLAN 99 Management Plane Isolation (L2 Split Post-L3 Conversion)

6.1.1 Problem Description

Right after we converted the distribution uplinks to routed /30 interfaces, **VM-AUTO** (10.99.99.100), sitting on **SW-CORE**, lost SSH reachability to every distribution switch (.11, .12, .13) and access switch (.21, .22, .23).

6.1.2 Root Cause Analysis

The routed uplinks had effectively cut VLAN 99 into **four separate Layer 2 islands**, one per switch. ARP requests from VM-AUTO for something like 10.99.99.11 never made it past SW-CORE, because there was no longer a bridged path over the uplink to carry them. On top of that, the access switches were still pointing their default gateway at 10.99.99.1 (SW-CORE), which they could no longer reach at Layer 2 either.

Listing 5: VLAN 99 Layer 2 Segmentation Diagram

ISLAND 1 (L2)			ISLAND 2 (L2)		
+-----+			+-----+		
VM-AUTO 10.99.99.100			SW-D-DEIE 10.99.99.11		
SW-Core 10.99.99.1	--ROUTED--	SW-A-DEIE 10.99.99.21			
SW-A-DIS 10.99.99.24	10.0.10				
+-----+			+-----+		
	/30				

6.1.3 How We Fixed It

1. **Host static routes on SW-CORE:** we added explicit /32 routes so management traffic bound for a specific switch would go out the correct /30 uplink instead of trying (and failing) to use the local /24:

```
ip route 10.99.99.11 255.255.255.255 10.0.10.2
ip route 10.99.99.21 255.255.255.255 10.0.10.2
ip route 10.99.99.12 255.255.255.255 10.0.20.2
ip route 10.99.99.22 255.255.255.255 10.0.20.2
ip route 10.99.99.13 255.255.255.255 10.0.30.2
ip route 10.99.99.23 255.255.255.255 10.0.30.2
```

2. **Return routes on the distribution switches:** added a host route back to 10.99.99.100 via SW-CORE's point-to-point interface, so replies could find their way home.

3. **Re-pointed the access switch gateways:** instead of 10.99.99.1, each access switch now points at its own local distribution switch (10.99.99.11, .12, or .13), which it can actually reach at Layer 2.

6.2 OSPF Adjacency Failure & ACL Filtering Remediation

6.2.1 Problem Description

OSPF wouldn't come up between **SW-CORE** and **R-CORE**, and separately, **VM-AUTO** started getting "Communication administratively prohibited" back whenever it pinged router interfaces.

6.2.2 Root Cause Analysis

Two things, as it turned out:

1. `passive-interface default` on SW-CORE was silently suppressing OSPF Hellos on Gi0/1 — we'd forgotten to exempt that interface.
2. `ACL-MGMT-IN` on SW-CORE had no explicit `permit icmp` entries for the transit subnets (10.0.0.0/30, 10.0.1.0/30), so the implicit deny at the end was catching pings that should have gone through.

6.2.3 How We Fixed It

1. Ran no `passive-interface GigabitEthernet0/1` under `router ospf 1` on SW-CORE, which let the Hellos through and OSPF formed immediately.
2. Rebuilt `ACL-MGMT-IN` to explicitly permit ICMP and SSH across the transit subnets before the implicit deny takes over.

7 References

- [1] Cisco Systems, *Designing Campus Three-Tier Hierarchical Architectures*, Cisco Press, 2023.
- [2] Moy, J., *OSPF Version 2*, RFC 2328, IETF, 1998.
- [3] Red Hat Inc., *Ansible Automation Platform Documentation*, 2024.
- [4] Netmiko Project, *Multi-vendor library for network automation CLI access*, Python Software Foundation, 2024.
- [5] Zabbix LLC, *Zabbix 6.0 LTS Enterprise Network Monitoring Manual*, 2024.

8 Appendices

8.1 Appendix A: Python Netmiko Script (01_configure_routers.py)

Listing 6: 01_configure_routers.py

```
#!/usr/bin/env python3
"""
EE8203_Campus_Network_Project_-_Netmiko_Router_Automation
Configures R-CORE and R-EDGE interfaces, OSPF Area 0, static routes, and NAT
overload.
```

```

"""
import yaml
import logging
from netmiko import ConnectHandler

logging.basicConfig(level=logging.INFO, format='%(asctime)s_%(levelname)s_%(message)s')

def load_inventory(filepath="inventory.yaml"):
    with open(filepath, 'r') as f:
        return yaml.safe_load(f)

def run_automation():
    inventory = load_inventory()
    for dev_name, params in inventory['routers'].items():
        logging.info(f"Applying configuration to {dev_name}...")
        try:
            conn = ConnectHandler(**params['connection'])
            conn.enable()
            output = conn.send_config_set(params['config'])
            logging.info(f"Output for {dev_name}: \n{output}")
            conn.save_config()
            conn.disconnect()
        except Exception as e:
            logging.error(f"Failed to configure {dev_name}: {e}")

if __name__ == "__main__":
    run_automation()

```

8.2 Appendix B: Ansible Site Playbook (site.yml)

Listing 7: site.yml

```

---
# EE8203 Master Switch Orchestration Playbook
- name: Phase 1 - Configure Global VLANs
  hosts: all_switches
  gather_facts: no
  roles:
    - vlans

- name: Phase 2 - Provision 802.1Q Trunks
  hosts: all_switches
  gather_facts: no
  roles:
    - trunking

- name: Phase 3 - Provision Access Ports & Portfast
  hosts: access_switches
  gather_facts: no
  roles:
    - access_ports

- name: Phase 4 - Provision Rapid-PVST+ STP
  hosts: all_switches
  gather_facts: no
  roles:
    - stp

- name: Phase 5 - Layer 3 Multilayer Distribution Setup
  hosts: dist_switches
  gather_facts: no
  roles:
    - l3_distribution

```

```

- name: Phase 6 - Access Switch Gateway Setup
  hosts: access_switches
  gather_facts: no
  roles:
    - l2_gateway

```

8.3 Appendix C: Sample Switch Configuration Extract (SW-D-DEIE)

```

hostname SW-D-DEIE
!
ip routing
!
vlan 10
  name VLAN_DEIE
vlan 99
  name MGMT
vlan 100
  name NATIVE
!
interface GigabitEthernet0/0
  description Routed Uplink to SW-CORE
  no switchport
  ip address 10.0.10.2 255.255.255.252
  ip ospf 1 area 0
!
interface GigabitEthernet0/2
  description Trunk to SW-A-DEIE
  switchport mode trunk
  switchport trunk native vlan 100
  switchport trunk allowed vlan 10,99,100
!
interface Vlan10
  ip address 10.10.10.1 255.255.255.0
  ip access-group ACL-DEIE-IN in
!
interface Vlan99
  ip address 10.99.99.11 255.255.255.0
!
router ospf 1
  router-id 3.3.3.11
  passive-interface default
  no passive-interface GigabitEthernet0/0
  network 10.0.10.0 0.0.0.3 area 0
  network 10.10.10.0 0.0.0.255 area 0
!
ip route 10.99.99.100 255.255.255.255 10.0.10.1
!
ip access-list extended ACL-DEIE-IN
  10 permit ip 10.10.10.0 0.0.0.255 10.10.40.0 0.0.0.255
  20 deny ip 10.10.10.0 0.0.0.255 10.10.20.0 0.0.0.255
  30 deny ip 10.10.10.0 0.0.0.255 10.10.30.0 0.0.0.255
  40 permit ip 10.10.10.0 0.0.0.255 any

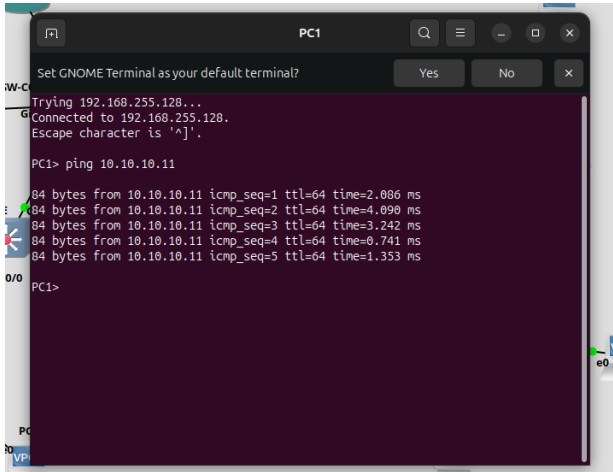
```

Listing 8: SW-D-DEIE Configuration Excerpt

8.4 Appendix D: Empirical 4×4 Department Reachability Test Evidence Screenshots

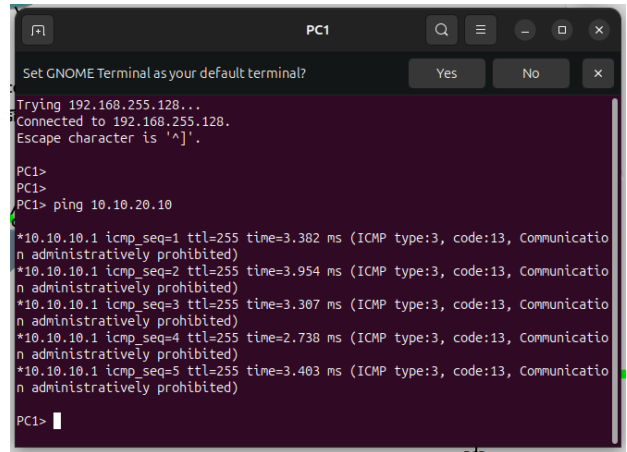
This appendix contains all 16 terminal screenshots from GNS3, one per cell of the 4×4 reachability matrix in Section 3.2.

8.4.1 Row 1: DEIE Host Reachability Evidence (Source IP: 10.10.10.10)



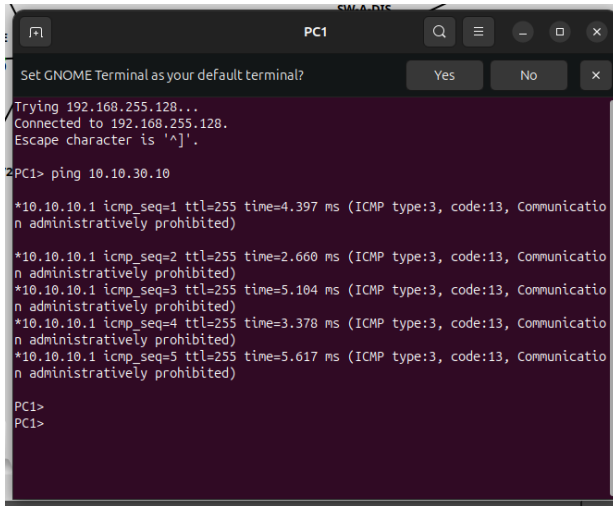
```
PC1
Set GNOME Terminal as your default terminal? [Yes/No]
Trying 192.168.255.128...
Connected to 192.168.255.128.
Escape character is '^['.
PC1> ping 10.10.10.11
84 bytes from 10.10.10.11 icmp_seq=1 ttl=64 time=2.086 ms
84 bytes from 10.10.10.11 icmp_seq=2 ttl=64 time=4.090 ms
84 bytes from 10.10.10.11 icmp_seq=3 ttl=64 time=3.242 ms
84 bytes from 10.10.10.11 icmp_seq=4 ttl=64 time=0.741 ms
84 bytes from 10.10.10.11 icmp_seq=5 ttl=64 time=1.353 ms
PC1>
```

(a) Cell [1,1]: DEIE → DEIE (✓ PASS)



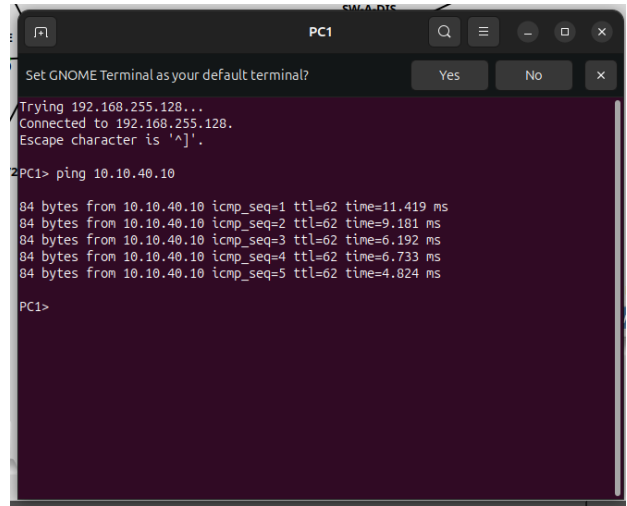
```
PC1
Set GNOME Terminal as your default terminal? [Yes/No]
Trying 192.168.255.128...
Connected to 192.168.255.128.
Escape character is '^['.
PC1>
PC1> ping 10.10.20.10
*10.10.10.1 icmp_seq=1 ttl=255 time=3.302 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.10.1 icmp_seq=2 ttl=255 time=3.954 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.10.1 icmp_seq=3 ttl=255 time=3.307 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.10.1 icmp_seq=4 ttl=255 time=2.730 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.10.1 icmp_seq=5 ttl=255 time=3.403 ms (ICMP type:3, code:13, Communication administratively prohibited)
PC1>
```

(b) Cell [1,2]: DEIE → DCEE (✗ FAIL)



```
PC1
Set GNOME Terminal as your default terminal? [Yes/No]
Trying 192.168.255.128...
Connected to 192.168.255.128.
Escape character is '^['.
PC1> ping 10.10.30.10
*10.10.10.1 icmp_seq=1 ttl=255 time=4.397 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.10.1 icmp_seq=2 ttl=255 time=2.660 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.10.1 icmp_seq=3 ttl=255 time=5.104 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.10.1 icmp_seq=4 ttl=255 time=3.378 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.10.1 icmp_seq=5 ttl=255 time=5.617 ms (ICMP type:3, code:13, Communication administratively prohibited)
PC1>
PC1>
```

(c) Cell [1,3]: DEIE → DMME (✗ FAIL)



```
PC1
Set GNOME Terminal as your default terminal? [Yes/No]
Trying 192.168.255.128...
Connected to 192.168.255.128.
Escape character is '^['.
PC1> ping 10.10.40.10
84 bytes from 10.10.40.10 icmp_seq=1 ttl=62 time=11.419 ms
84 bytes from 10.10.40.10 icmp_seq=2 ttl=62 time=9.181 ms
84 bytes from 10.10.40.10 icmp_seq=3 ttl=62 time=6.192 ms
84 bytes from 10.10.40.10 icmp_seq=4 ttl=62 time=6.733 ms
84 bytes from 10.10.40.10 icmp_seq=5 ttl=62 time=4.824 ms
PC1>
```

(d) Cell [1,4]: DEIE → DIS (✓ PASS)

Figure 4: DEIE Source Terminal Screenshots (Row 1 of Department Reachability Matrix)

8.4.2 Row 2: DCEE Host Reachability Evidence (Source IP: 10.10.20.10)

8.4.3 Row 3: DMME Host Reachability Evidence (Source IP: 10.10.30.10)

8.4.4 Row 4: DIS Host Reachability Evidence (Source IP: 10.10.40.10)

```

Set GNOME Terminal as your default terminal? [Yes/No]
Trying 192.168.255.128...
Connected to 192.168.255.128.
Escape character is '^J'.
PC3> ping 10.10.10.10

*10.10.20.1 icmp_seq=1 ttl=255 time=3.707 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.20.1 icmp_seq=2 ttl=255 time=3.970 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.20.1 icmp_seq=3 ttl=255 time=2.939 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.20.1 icmp_seq=4 ttl=255 time=3.202 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.20.1 icmp_seq=5 ttl=255 time=2.900 ms (ICMP type:3, code:13, Communication administratively prohibited)

PC3>

```

(a) Cell [2,1]: DCEE → DEIE (× FAIL)

```

Set GNOME Terminal as your default terminal? [Yes/No]
Trying 192.168.255.128...
Connected to 192.168.255.128.
Escape character is '^J'.
PC3> ping 10.10.20.11

84 bytes from 10.10.20.11 icmp_seq=1 ttl=64 time=4.180 ms
84 bytes from 10.10.20.11 icmp_seq=2 ttl=64 time=0.936 ms
84 bytes from 10.10.20.11 icmp_seq=3 ttl=64 time=2.032 ms
84 bytes from 10.10.20.11 icmp_seq=4 ttl=64 time=1.724 ms
84 bytes from 10.10.20.11 icmp_seq=5 ttl=64 time=2.178 ms

PC3>

```

(b) Cell [2,2]: DCEE → DCEE (✓ PASS)

```

Set GNOME Terminal as your default terminal? [Yes/No]
Trying 192.168.255.128...
Connected to 192.168.255.128.
Escape character is '^J'.
PC3> ping 10.10.30.10

*10.10.20.1 icmp_seq=1 ttl=255 time=2.906 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.20.1 icmp_seq=2 ttl=255 time=2.617 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.20.1 icmp_seq=3 ttl=255 time=2.166 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.20.1 icmp_seq=4 ttl=255 time=3.461 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.20.1 icmp_seq=5 ttl=255 time=4.068 ms (ICMP type:3, code:13, Communication administratively prohibited)

PC3>

```

(c) Cell [2,3]: DCEE → DMME (× FAIL)

```

Set GNOME Terminal as your default terminal? [Yes/No]
Trying 192.168.255.128...
Connected to 192.168.255.128.
Escape character is '^J'.
PC3> ping 10.10.40.10

*10.10.20.1 icmp_seq=1 ttl=255 time=6.330 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.20.1 icmp_seq=2 ttl=255 time=3.018 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.20.1 icmp_seq=3 ttl=255 time=14.638 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.20.1 icmp_seq=4 ttl=255 time=5.810 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.20.1 icmp_seq=5 ttl=255 time=2.954 ms (ICMP type:3, code:13, Communication administratively prohibited)

PC3>

```

(d) Cell [2,4]: DCEE → DIS (Ping × FAIL, HTTP Web Only)

Figure 5: DCEE Source Terminal Screenshots (Row 2 of Department Reachability Matrix)

```

Set GNOME Terminal as your default terminal? [Yes] [No] [X]
Trying 192.168.255.128...
Connected to 192.168.255.128.
Escape character is '^J'.
PC5> ping 10.10.10.10

*10.10.30.1 icmp_seq=1 ttl=255 time=4.105 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.30.1 icmp_seq=2 ttl=255 time=3.694 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.30.1 icmp_seq=3 ttl=255 time=4.020 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.30.1 icmp_seq=4 ttl=255 time=2.358 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.30.1 icmp_seq=5 ttl=255 time=4.464 ms (ICMP type:3, code:13, Communication administratively prohibited)
PC5>

```

(a) Cell [3,1]: DMME → DEIE (× FAIL)

```

Set GNOME Terminal as your default terminal? [Yes] [No] [X]
Trying 192.168.255.128...
Connected to 192.168.255.128.
Escape character is '^J'.
PC5> ping 10.10.20.10

*10.10.30.1 icmp_seq=1 ttl=255 time=2.479 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.30.1 icmp_seq=2 ttl=255 time=2.982 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.30.1 icmp_seq=3 ttl=255 time=2.482 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.30.1 icmp_seq=4 ttl=255 time=3.761 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.30.1 icmp_seq=5 ttl=255 time=2.608 ms (ICMP type:3, code:13, Communication administratively prohibited)
PC5>

```

(b) Cell [3,2]: DMME → DCEE (× FAIL)

```

Set GNOME Terminal as your default terminal? [Yes] [No] [X]
Trying 192.168.255.128...
Connected to 192.168.255.128.
Escape character is '^J'.
PC5> ping 10.10.30.11

84 bytes from 10.10.30.11 icmp_seq=1 ttl=64 time=0.901 ms
84 bytes from 10.10.30.11 icmp_seq=2 ttl=64 time=1.237 ms
84 bytes from 10.10.30.11 icmp_seq=3 ttl=64 time=2.125 ms
84 bytes from 10.10.30.11 icmp_seq=4 ttl=64 time=0.834 ms
84 bytes from 10.10.30.11 icmp_seq=5 ttl=64 time=1.356 ms
PC5>

```

(c) Cell [3,3]: DMME → DMME (✓ PASS)

```

Set GNOME Terminal as your default terminal? [Yes] [No] [X]
Trying 192.168.255.128...
Connected to 192.168.255.128.
Escape character is '^J'.
PC5> ping 10.10.40.10

*10.10.30.1 icmp_seq=1 ttl=255 time=2.872 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.30.1 icmp_seq=2 ttl=255 time=2.938 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.30.1 icmp_seq=3 ttl=255 time=2.619 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.30.1 icmp_seq=4 ttl=255 time=2.398 ms (ICMP type:3, code:13, Communication administratively prohibited)
*10.10.30.1 icmp_seq=5 ttl=255 time=5.509 ms (ICMP type:3, code:13, Communication administratively prohibited)
PC5>

```

(d) Cell [3,4]: DMME → DIS (× FAIL)

Figure 6: DMME Source Terminal Screenshots (Row 3 of Department Reachability Matrix)

```

Set GNOME Terminal as your default terminal? [Yes] [No] [X]
Trying 192.168.255.128...
Connected to 192.168.255.128.
Escape character is '^['.

PC8> ping 10.10.10.10

84 bytes from 10.10.10.10 icmp_seq=1 ttl=62 time=9.565 ms
84 bytes from 10.10.10.10 icmp_seq=2 ttl=62 time=10.084 ms
84 bytes from 10.10.10.10 icmp_seq=3 ttl=62 time=6.075 ms
84 bytes from 10.10.10.10 icmp_seq=4 ttl=62 time=8.395 ms
84 bytes from 10.10.10.10 icmp_seq=5 ttl=62 time=7.745 ms

PC8>

```

(a) Cell [4,1]: DIS → DEIE (✓ PASS)

```

Set GNOME Terminal as your default terminal? [Yes] [No] [X]
Trying 192.168.255.128...
Connected to 192.168.255.128.
Escape character is '^['.

PC8> ping 10.10.20.10

*10.10.40.1 icmp_seq=1 ttl=255 time=3.938 ms (ICMP type:3, code:13, Communicatio
n administratively prohibited)
*10.10.40.1 icmp_seq=2 ttl=255 time=3.228 ms (ICMP type:3, code:13, Communicatio
n administratively prohibited)
*10.10.40.1 icmp_seq=3 ttl=255 time=3.409 ms (ICMP type:3, code:13, Communicatio
n administratively prohibited)
*10.10.40.1 icmp_seq=4 ttl=255 time=2.838 ms (ICMP type:3, code:13, Communicatio
n administratively prohibited)
*10.10.40.1 icmp_seq=5 ttl=255 time=3.177 ms (ICMP type:3, code:13, Communicatio
n administratively prohibited)

PC8>

```

(b) Cell [4,2]: DIS → DCEE (Ping ✗ FAIL, Established TCP)

```

Set GNOME Terminal as your default terminal? [Yes] [No] [X]
Trying 192.168.255.128...
Connected to 192.168.255.128.
Escape character is '^['.

PC8> ping 10.10.30.10

*10.10.40.1 icmp_seq=1 ttl=255 time=3.223 ms (ICMP type:3, code:13, Communicatio
n administratively prohibited)
*10.10.40.1 icmp_seq=2 ttl=255 time=2.463 ms (ICMP type:3, code:13, Communicatio
n administratively prohibited)
*10.10.40.1 icmp_seq=3 ttl=255 time=2.504 ms (ICMP type:3, code:13, Communicatio
n administratively prohibited)
*10.10.40.1 icmp_seq=4 ttl=255 time=4.602 ms (ICMP type:3, code:13, Communicatio
n administratively prohibited)
*10.10.40.1 icmp_seq=5 ttl=255 time=2.513 ms (ICMP type:3, code:13, Communicatio
n administratively prohibited)

PC8>

```

(c) Cell [4,3]: DIS → DMME (✗ FAIL)

```

Set GNOME Terminal as your default terminal? [Yes] [No] [X]
Trying 192.168.255.128...
Connected to 192.168.255.128.
Escape character is '^['.

PC7> ping 10.10.40.11

84 bytes from 10.10.40.11 icmp_seq=1 ttl=64 time=2.245 ms
84 bytes from 10.10.40.11 icmp_seq=2 ttl=64 time=3.568 ms
84 bytes from 10.10.40.11 icmp_seq=3 ttl=64 time=1.990 ms
84 bytes from 10.10.40.11 icmp_seq=4 ttl=64 time=2.013 ms
84 bytes from 10.10.40.11 icmp_seq=5 ttl=64 time=1.986 ms

PC7>

```

(d) Cell [4,4]: DIS → DIS (✓ PASS)

Figure 7: DIS Source Terminal Screenshots (Row 4 of Department Reachability Matrix)